

대기 등록 API

 매장에 대기 등록

대기 등록 API

요청 경로

1 | POST /v1/restaurants/:restaurantIdx/reservations

- restaurantIdx :매장 idx

요청 헤더

이름	값	설명
Content-Type	application/json	
Authorization	Bearer {{token}}	로그인 api 를 통해서 받은 토큰

요청 바디

이름		값	설명	필수 여부	비고
guest		object	고객 정보	V	
	name	string	이름		
	phone	string	전화번호		
	note	string	고객 메모		
	isMarketing	boolean	마케팅 동의		
	isThirdParty	boolean	제3자 정보제공 동의		
	isPartnerInfoAgreement	boolean	제휴사 광고 개인 정보 수집 및 이용 동의 여부		☒ TB-13105: [BE][키오스크] 대기 생성 시, 고객의 제

	isPartnerAdAgreement	boolean	제휴사 광고성 정보 수신 동의 여부		휴사 광고 수 신 관련 필드 추가 리뷰 중
	email	string	이메일		
	locale	string	언어		
reservation		object	대기 정보	V	
	numOfPeople	number	인원	V	
	childCount	number	아동 인원		
	memo	string	대기 메모		
	queueIndex	number	대기열 인덱스	V	대기열 사용하 지 않을 때: -1
menus		Array<object>	메뉴 정보		
	_id	string	메뉴의 id	V	
	name	string	메뉴명		
	price	string	메뉴 가격		
	isMain	boolean	메인 여부		
	isSoldOut	boolean	매진 여부		
	quantity	number	개수	V	
	options	Array<object>	옵션 목록		
		name	string	옵션명	V
		price	number	가격	V
		isSoldOut	boolean	품절여부	V
		_id	string	옵션 id	V
		quantity	number	옵션 수량	default = 1
	size	object	사이즈	V	
		name	string	사이즈명	V

		price	number	사이즈 가격	V	
		_id	string	사이즈 id	V	

요청 예시

```

1 test 환경 / restaurantIdx: 22
2 {
3   "guest": {
4     "name": "테스트 고객",
5     "phone": "01000001234",
6     "note": "테스트 메모 (kiosk) isThirdParty(false)",
7     "isMarketing": true,
8     "isThirdParty": false
9   },
10  "reservation": {
11    "numOfPeople": 2,
12    "memo": "isMarketing(true) isThirdParty(false) 테스트 고객 01000001234 kiosk server 김밥 2천원 2개 주문",
13    "queueIndex": 1,
14  },
15  "menus": [
16    {
17      "_id": "6135d1898ffbb15296197362",
18      "name": "김밥",
19      "price": 2000,
20      "isMain": false,
21      "isSoldOut": false,
22      "quantity": 2,
23      "options": [],
24      "size": {
25        "name": "기 본",
26        "price": 2000,
27        "_id": "6135d1a5851e90095266de48"
28      }
29    }
30  ]
31 }

```

요청 예시

```

1 test 환경 / restaurantIdx: 51704 / 이메일만 발송 / 주문 없음 / 대기열 세분화 없음
2 {
3   "guest": {
4     "name": "1130",
5     "email": "test@mealant.com",
6     "note": "test",
7     "isMarketing": true,
8     "locale": "en",
9     "isThirdParty": false
10  },
11  "reservation": {
12    "numOfPeople": 2,
13    "memo": "test"
14  }
15 }

```

응답 바디

이름	값	설명	비고
idx	number	대기 idx	
hashKey	string	고유 해시 값	
queueName	string	대기열명	대기열 사용하지 않을 때: "" (empty string)
waitSequenceStatic	number	대기 번호	

요청 성공

```

1 HTTP/1.1 201 Created
2 {
3   "idx": 36315,
4   "hashKey": "1su28a4kj76w343ih2wea",
5   "queueName": "포장",
6   "waitSequenceStatic": 3
7 }
```

요청 실패

▽ 정책 위반 에러

전화번호가 중복인 경우

```

1 HTTP/1.1 406 Not Acceptable
2 {
3   "error": {
4     "status": 406,
5     "name": "PolicyError",
6     "code": "DUPLICATED_PHONE",
7     "message": "중복된 전화번호 입니다.",
8     "data": {
9       "idx": 23055,
10      "guest": {
11        "idx": 277258,
12        "phone": "00012345678"
13      }
14    }
15  }
16 }
```

대기열 미사용 매장의 대기가 마감 상태인 경우

```

1 HTTP/1.1 406 Not Acceptable
2 {
3   "error": {
4     "status": 406,
5     "name": "PolicyError",
6     "code": "NOT_SERVICE_RESTAURANT",
7     "message": "마감된 매장입니다."
8   }
9 }
```

대기열 사용 매장의 대기열이 마감 상태인 경우

```

1 HTTP/1.1 406 Not Acceptable
2 {
3   "error": {
4     "status": 406,
5     "name": "PolicyError",
6     "code": "CLOSED_WAITING_QUEUE",
7     "message": "마감된 대기입니다."
8   }
9 }

```

▽ 매장 옵션 기반 입력값 검증 에러

현장대기 계약 (contracts.waiting.useKiosk) 이 Off 인 경우

```

1 HTTP/1.1 406 Not Acceptable
2 {
3   "error": {
4     "status": 406,
5     "name": "PolicyError",
6     "code": "NOT_CONTRACT_RESTAURANT",
7     "message": "계약이 되지 않은 매장입니다."
8   }
9 }

```

매장의 옵션중 이름이 필수로 설정이 되어있으나 요청에 이름이 없는 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "RequiredError",
6     "code": "REQUIRED_NAME",
7     "message": "이름 필수 매장입니다."
8   }
9 }

```

매장의 옵션중 전화번호가 필수로 설정이 되어있으나 요청에 전화번호가 없는 경우 (isPhoneNumber 도 false 인경우)

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "RequiredError",
6     "code": "REQUIRED_PHONE_NUMBER",
7     "message": "전화번호 필수 매장입니다."
8   }
9 }

```

매장의 옵션 중 최대 가능 인원이 설정 되어있으나, 요청에 인원수가 최대 가능 인원을 초과하는 경우

```

1 HTTP/1.1 406 Not Acceptable
2 {
3   "error": {
4     "status": 406,
5     "name": "PolicyError",
6     "code": "OVER_NUM_OF_PEOPLE",
7     "message": "대기 가능 인원을 초과 했습니다."
8   }
9 }

```

매장의 옵션에 유아 인원 설정 안되어있으나, 요청에 유아 인원수가 있는 경우

```

1 HTTP/1.1 406 Not Acceptable
2 {
3   "error": {
4     "status": 406,

```

```

5     "name": "PolicyError",
6     "code": "NOT_USE_CHILD_COUNT",
7     "message": "유아 인원을 사용하지 않는 매장입니다."
8   }
9 }

```

매장의 옵션에 메뉴를 받는다고 설정이 안되어있으나, 요청에 메뉴가 있는 경우

```

1 HTTP/1.1 406 Not Acceptable
2 {
3   "error": {
4     "status": 406,
5     "name": "PolicyError",
6     "code": "NOT_USE_ORDER_MENU",
7     "message": "메뉴 주문을 사용하지 않는 매장입니다."
8   }
9 }

```

▼ 주문 메뉴 검증 에러

메뉴 필수 매장에서 주문 메뉴가 빈 배열일 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "BadRequestError",
6     "code": "PLEASE_SELECT_MENU",
7     "message": "메뉴 선택 후 다시 시도해 주세요."
8   }
9 }

```

주문 메뉴에 성인 메뉴가 있는 경우

```

1 HTTP/1.1 406 Not Acceptable
2 {
3   "error": {
4     "status": 406,
5     "name": "PolicyError",
6     "code": "BAD_ORDER_MENU",
7     "message": "주문할 수 없는 메뉴가 있습니다. 메뉴 선택 화면에서 확인해 주세요."
8   }
9 }

```

주문한 메뉴중 메인 메뉴의 수량이 초과한 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "BadRequestError",
6     "code": "PLEASE_CHECK_AVAILABLE_QUANTITY",
7     "message": "메인 메뉴가 주문 가능한 수량을 초과했습니다. 주문 가능 수량 : {orderAvailableQuantity}"
8   }
9 }

```

주문할 수 없는 메뉴가 있는 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "BadRequestError",
6     "code": "BAD_ORDER_MENU",
7     "message": "주문할 수 없는 메뉴가 있습니다. 장바구니에서 확인해 주세요."
8   }

```

```
9 }
```

주문한 메뉴 중 한정 메뉴의 수량이 초과한 경우

```
1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "BadRequestError",
6     "code": "PLEASE_CHECK_LIMIT_MENU_AVAILABLE_QUANTITY",
7     "message": "한정 메뉴가 주문 가능한 수량을 초과했습니다."
8   }
9 }
```

주문한 메뉴 중 품절 메뉴가 포함된 경우

```
1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "BadRequestError",
6     "code": "SOLD_OUT_MENU",
7     "message": "품절로 주문할 수 없는 메뉴가 있습니다. 메뉴 선택 화면에서 수정 후 다시 주문해 주세요."
8   }
9 }
```

주문한 메뉴 중 수량 부족으로 주문할 수 없는 메뉴가 있는 경우

```
1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "BadRequestError",
6     "code": "DAILY_OVER_QUANTITY",
7     "message": "수량 부족으로 주문할 수 없는 메뉴가 있습니다. 장바구니에서 수정 후 다시 주문해 주세요."
8   }
9 }
```

✓ 요청 바디 검증

매장 idx 가 숫자가 아닌 경우

```
1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_RESTAURANTIDX",
7     "message": "매장 idx(restaurantIdx)가 입력형식에 맞지 않습니다."
8   }
9 }
```

매장 idx 가 1보다 작은 경우

```
1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "BadRequestError",
6     "code": "MIN_RESTAURANTIDX",
7     "message": "매장 idx(restaurantIdx)가 가능한 범위가 아닙니다."
8   }
9 }
```

매장 idx 가 숫자가 아닌 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_RESTAURANTIDX",
7     "message": "매장 idx(restaurantIdx)가 입력형식에 맞지 않습니다."
8   }
9 }

```

childCount 가 숫자가 아닌 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_CHILDCOUNT",
7     "message": "유아 인원수(childCount)가 입력형식에 맞지 않습니다."
8   }
9 }

```

인원수가 0명 이하인 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "BadRequestError",
6     "code": "MIN_NUMOFPEOPLE",
7     "message": "성인 인원수(numOfPeople)가 가능한 범위가 아닙니다."
8   }
9 }

```

인원수가 숫자가 아닌 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_NUMOFPEOPLE",
7     "message": "성인 인원수(numOfPeople)가 입력형식에 맞지 않습니다."
8   }
9 }

```

인원수가 없는 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "RequiredError",
6     "code": "REQUIRED_NUMOFPEOPLE",
7     "message": "성인 인원수(numOfPeople)의 값은 필수입니다."
8   }
9 }

```

대기 메모가 문자열이 아닌 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_MEMO",

```

```

7     "message": "대기 메모(memo)가 입력형식에 맞지 않습니다."
8   }
9 }

```

고객의 이름이 문자열이 아닌 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_NAME",
7     "message": "고객 이름(name)가 입력형식에 맞지 않습니다."
8   }
9 }

```

전화번호가 문자열이 아닌 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_PHONE",
7     "message": "고객 전화번호(phone)가 입력형식에 맞지 않습니다."
8   }
9 }

```

고객 메모가 문자열이 아닌 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_NOTE",
7     "message": "고객 메모(note)가 입력형식에 맞지 않습니다."
8   }
9 }

```

마케팅 동의 여부가 boolean 값이 아닌 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_ISMARKETING",
7     "message": "고객 마케팅 동의 여부(isMarketing)는 true/false로 입력해주세요."
8   }
9 }

```

제 3자 정보제공 동의 여부 가 boolean 값이 아닌 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_ISTHIRDPARTY",
7     "message": "고객 제 3자 정보제공 동의 여부(isThirdParty)는 true/false로 입력해주세요."
8   }
9 }

```

메뉴의 _id 값이 ObjectId 형식에 맞지 않는 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "BadRequestError",
6     "code": "PATTERN__ID",
7     "message": "메뉴 id가 ObjectId 형식에 맞지 않습니다."
8   }
9 }

```

메뉴의 _id 값이 없는 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "RequiredError",
6     "code": "REQUIRED__ID",
7     "message": "메뉴 id(_id)의 값은 필수입니다."
8   }
9 }

```

주문의 개수가 없는 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "RequiredError",
6     "code": "REQUIRED_QUANTITY",
7     "message": "주문 개수(quantity)의 값은 필수입니다."
8   }
9 }

```

주문의 개수가 숫자가 아닌 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_QUANTITY",
7     "message": "주문 개수(quantity)가 입력형식에 맞지 않습니다."
8   }
9 }

```

메뉴의 명이 문자열이 아닌 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_NAME",
7     "message": "메뉴 명(name)가 입력형식에 맞지 않습니다."
8   }
9 }

```

메뉴의 가격이 숫자가 아닌 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_PRICE",

```

```
7     "message": "메뉴 가격(price)가 입력형식에 맞지 않습니다."
8   }
9 }
```

메뉴 메인 여부가 boolean 값이 아닌 경우

```
1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_ISMAIN",
7     "message": "메뉴 메인 여부(isMain)는 true/false로 입력해주세요."
8   }
9 }
```

메뉴 매진 여부가 boolean 값이 아닌 경우

```
1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_ISSOLDOUT",
7     "message": "메뉴 매진 여부(isSoldOut)는 true/false로 입력해주세요."
8   }
9 }
```

옵션의 명이 문자열이 아닌 경우

```
1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_NAME",
7     "message": "메뉴 옵션 명(name)가 입력형식에 맞지 않습니다."
8   }
9 }
```

옵션의 가격이 숫자가 아닌 경우

```
1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_PRICE",
7     "message": "메뉴 옵션 가격(price)가 입력형식에 맞지 않습니다."
8   }
9 }
```

옵션의 _id 값이 ObjectId 형식에 맞지 않는 경우

```
1 {
2   "error": {
3     "status": 400,
4     "name": "BadRequestError",
5     "code": "PATTERN__ID",
6     "message": "옵션 id가 ObjectId 형식에 맞지 않습니다."
7   }
8 }
```

옵션의 _id 값이 없는 경우

```
1 HTTP/1.1 400 Bad Request
```

```

2 {
3   "error": {
4     "status": 400,
5     "name": "RequiredError",
6     "code": "REQUIRED__ID",
7     "message": "옵션 id(_id)의 값은 필수입니다."
8   }
9 }

```

사이즈가 없는 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "RequiredError",
6     "code": "REQUIRED_SIZE",
7     "message": "메뉴 사이즈(size)의 값은 필수입니다."
8   }
9 }

```

사이즈의 명이 문자열이 아닌 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_NAME",
7     "message": "메뉴 사이즈 명(name)가 입력형식에 맞지 않습니다."
8   }
9 }

```

사이즈의 가격이 숫자가 아닌 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "DataTypeError",
6     "code": "BASE_PRICE",
7     "message": "메뉴 사이즈 가격(price)가 입력형식에 맞지 않습니다."
8   }
9 }

```

사이즈의 _id 값이 ObjectId 형식에 맞지 않는 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "BadRequestError",
6     "code": "PATTERN__ID",
7     "message": "사이즈 id가 ObjectId 형식에 맞지 않습니다."
8   }
9 }

```

사이즈의 _id 값이 없는 경우

```

1 HTTP/1.1 400 Bad Request
2 {
3   "error": {
4     "status": 400,
5     "name": "RequiredError",
6     "code": "REQUIRED__ID",
7     "message": "사이즈 id(_id)의 값은 필수입니다."
8   }
9 }

```

```
8 }  
9 }
```

버전	날짜	댓글
현재 버전 (v. 12)	2026-4월-09-06:50	김세현
v. 11	2025-7월-23-06:35	손유진
v. 10	2024-11월-05-04:10	조재영 menus.options.quantity 추가
v. 9	2023-8월-21-00:48	강성문
v. 8	2023-8월-21-00:46	강성문
v. 7	2023-5월-16-02:53	손유진 - 메뉴 필수 선택, 품질 메뉴에 대한 예러 추가 - 잘못 작성되어있던 메뉴 수량 부족에 의한 예러 코드 수정
v. 6	2023-4월-28-05:21	손유진 대기 등록 완료 후 응답 데이터 수정
v. 5	2023-4월-06-07:46	손유진 - 요청 바디에 queueIndex 추가 - 응답 바디에 queueIndex, queueName 추가 - 대기열 마감 시 반환하는 예러 추가
v. 4	2022-3월-22-09:00	강성문
v. 3	2022-1월-04-06:35	강성문
v. 2	2021-12월-21-00:56	강성문
v. 1	2021-12월-20-09:57	강성문